

RUSHIKESH'S INDIVIDUAL TASK SHEET

Role: Backend Developer (Team A - AI & Testing Module)

Team Partner: Aditya (Team Lead)

Timeline: 3 Months (90 Days)

Total Hours: 864 hours (72 hours/week × 12 weeks)

Working Days: Monday to Saturday (12 hours/day)

Rest Day: Sunday (Mandatory - NO CODING!)

YOUR PROFILE & RESPONSIBILITIES

Primary Focus Areas:

1. **Backend Development** (60% time)
 - User management and registration APIs
 - Test session management
 - Question processing and validation
 - Data import/export features
 - Supporting APIs for core features
2. **Testing & Quality** (20% time)
 - Unit testing for APIs
 - Integration testing
 - API documentation
 - Postman collection maintenance
3. **Support Features** (15% time)
 - File processing (CSV, Excel, images)
 - Template management
 - Scheduled tasks and cron jobs
 - Email/SMS notifications
4. **Learning & Growth** (5% time)
 - Learn new technologies
 - Research best practices
 - Code quality improvement

Your Strengths (Leverage These):

- Good coding fundamentals
- Quick learner
- Detail-oriented
- Problem-solving skills

What You'll Learn:

- Professional backend development (NestJS/Node.js)
- Supabase (Database + Auth + Storage)
- API design best practices
- Testing strategies
- Working in a fast-paced team



MONTH 1: FOUNDATION (WEEKS 1-4)

WEEK 1: SETUP & USER REGISTRATION (72 hours)

Your Focus: Learn Supabase, build registration APIs, support Aditya

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Environment Setup & Learning

Hour-by-Hour Breakdown:

- **Hours 1-4:** Setup local environment
 - [] Install Node.js (v20 LTS)
 - [] Install VS Code with extensions:
 - ESLint
 - Prettier
 - Thunder Client (API testing)
 - [] Install Git, configure SSH keys for GitHub
 - [] Clone repository created by Aditya
 - [] Install dependencies: `npm install`
- **Hours 5-10:** Learn Supabase basics
 - [] Watch Supabase crash course (YouTube - 2 hours)
 - [] Read Supabase Auth documentation (1 hour)
 - [] Read Supabase Database documentation (1 hour)
 - [] Practice basic queries in Supabase dashboard (1 hour)
 - [] Complete Supabase tutorial project (1 hour)
- **Hours 11-16:** Understand existing code

- ☐ Read Aditya's database schema
- ☐ Study authentication APIs created by Aditya
- ☐ Understand folder structure
- ☐ Run project locally, test auth APIs in Postman
- ☐ Ask Aditya questions (take notes)
- **Hours 17-24:** First API development
 - ☐ Task: Build user profile API
 - ☐ GET /api/users/:id (get user profile)
 - ☐ PATCH /api/users/:id (update profile)
 - ☐ Implement input validation (Zod)
 - ☐ Test in Postman
 - ☐ Create pull request, get Aditya's review

Deliverables Day 1-2:

- ☒ Local environment set up
 - ☒ Supabase basics understood
 - ☒ First API created and reviewed
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: User Registration System

- **Hours 1-8:** Student registration
 - ☐ POST /api/students/register
- ☐ Input validation:


```
{ name: string (required, min 2 chars), email: string (valid email), phone: string (10 digits), class: string, batch: string, parentPhone: string (10 digits)}
```

 - ☐ Hash password (bcrypt) - actually Supabase handles this
 - ☐ Create user in Supabase Auth
 - ☐ Create student record in students table
 - ☐ Return JWT token + user data
- **Hours 9-16:** Teacher & Parent registration
 - ☐ POST /api/teachers/register
 - ☐ POST /api/parents/register
 - ☐ Similar logic to student registration
 - ☐ Different fields for each role
 - ☐ Link parent to student (parent_id in students table)
- **Hours 17-24:** Email verification
 - ☐ Implement email verification flow

- ☐ Send verification email (Supabase handles this)
- ☐ POST /api/auth/verify-email
- ☐ Handle verification token
- ☐ Update user status to verified

Deliverables Day 3-4:

- ☒ Complete registration system for all user types
 - ☒ Email verification working
-

Day 5 (Friday) - 12 hours

Task: Password Management

- **Hours 1-6:** Password reset
 - ☐ POST /api/auth/forgot-password (send reset email)
 - ☐ POST /api/auth/reset-password (reset with token)
 - ☐ Validate token expiry (15 minutes)
 - ☐ Update password in Supabase Auth
- **Hours 7-12:** Role management
 - ☐ POST /api/users/:id/change-role (admin only)
 - ☐ Validate permissions (only admin can change roles)
 - ☐ Update user role in database
 - ☐ Test role-based access control

Deliverables Day 5:

- ☒ Password management complete
 - ☒ Role management working
-

Day 6 (Saturday) - 12 hours

Task: Testing & Documentation

- **Hours 1-6:** Unit testing
 - ☐ Learn Jest basics (1 hour)

☐ Write tests for registration APIs:

```
describe('POST /api/students/register', () => { it('should create a student', async () => { //
test code }); it('should return 400 for invalid email', async () => { // test code });});
```

-
- ☐ Run tests: `npm test`

- ☐ Aim for 70%+ code coverage
- **Hours 7-12:** API documentation
 - ☐ Document all your APIs in Postman
 - ☐ Add example requests and responses
 - ☐ Share collection with team
 - ☐ Update README with API endpoints

Week 1 Complete! 🎉

- Total Hours: 72 hours
- Integration: Registration APIs ready ☒

WEEK 2: QUESTION UPLOAD & PROCESSING (72 hours)

Your Focus: Build question upload and file processing features

Day 1-2 (Monday-Tuesday) - 24 hours

Task: CSV/Excel Upload Processing

- **Hours 1-8:** Learn file processing
 - ☐ Research CSV parsing libraries (PapaParse)
 - ☐ Research Excel parsing libraries (xlsx, exceljs)
 - ☐ Tutorial: Read CSV file in Node.js (1 hour)
 - ☐ Tutorial: Read Excel file in Node.js (1 hour)
- **Hours 9-18:** Build upload processor
 - ☐ POST /api/questions/import
 - ☐ Accept file upload (use multer middleware)

☐ Parse CSV file:

```
const Papa = require('papaparse'); Papa.parse(fileContent, { header: true, complete:
(results) => { // results.data = array of questions // validate and insert into database }});
```

- ☐ Parse Excel file using xlsx library
- ☐ Support both CSV and Excel (.xlsx, .xls)
- **Hours 19-24:** Data validation
 - ☐ Validate each row:
 - ☐ Required fields present (question_text, subject, chapter)
 - ☐ Data types correct (marks must be number)
 - ☐ Subject and chapter exist in database
 - ☐ Options format correct for MCQs
 - ☐ Collect errors for invalid rows

☐ Return summary:

```
{ "total": 500, "success": 490, "failed": 10, "errors": [ { "row": 25, "error": "Invalid subject 'Physiks' (should be Physics)" } ] }
```

○

Deliverables Day 1-2:

- ☒ File upload and parsing working
 - ☒ Data validation implemented
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Bulk Database Operations

- **Hours 1-10:** Bulk insert optimization
 - ☐ Don't insert one by one (slow!)

☐ Use batch insert (100 questions at a time):

```
await supabase .from('questions') .insert(questionsArray); // insert multiple rows
```

○

- ☐ Handle duplicate questions (check before insert)
- ☐ Use database transactions (all or nothing)
- **Hours 11-18:** Progress tracking
 - ☐ For large imports (1000+ questions), add progress tracking
 - ☐ Use Socket.io or Server-Sent Events (SSE)

☐ Send progress updates to frontend:

```
// Progress: 100/1000 questions inserted (10%)// Progress: 500/1000 questions inserted (50%)// Progress: 1000/1000 questions inserted (100%)
```

○

- **Hours 19-24:** Error handling & retry
 - ☐ If a batch fails, don't fail entire import
 - ☐ Retry failed batches (max 3 attempts)
 - ☐ Log errors to database for debugging
 - ☐ Test with 1000+ question import

Deliverables Day 3-4:

- ☒ Can import 1000+ questions in < 30 seconds
 - ☒ Proper error handling
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Question Templates & Export

- **Hours 1-12:** Question templates
 - ☐ Create question_templates table
 - ☐ POST /api/question-templates (save template)
 - ☐ GET /api/question-templates (list templates)
 - ☐ POST /api/questions/from-template (create from template)

☐ Example template:

```
{ "name": "Speed Calculation", "pattern": "{object} travels {distance} km in {time} hours. Find speed.", "variables": { "object": ["car", "bike", "train"], "distance": [60, 80, 100], "time": [2, 3, 4] }, "formula": "speed = distance / time" }
```

-
- **Hours 13-20:** Question export
 - ☐ GET /api/questions/export/csv
 - ☐ GET /api/questions/export/excel
 - ☐ Export filtered questions (by subject, chapter, etc.)
 - ☐ Download as file
- **Hours 21-24:** Week 2 wrap-up
 - ☐ Code review with Aditya
 - ☐ Fix any bugs found
 - ☐ Update documentation
 - ☐ Week 2 retrospective

Week 2 Complete! 🎉

- Integration: Question bank ready for testing ☒
-

WEEK 3: TEST SESSION SUPPORT (72 hours)

Your Focus: Build supporting features for test management

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Test Attempt Tracking

- **Hours 1-12:** Attempt validation
 - ☐ GET /api/tests/:testId/can-attempt
 - ☐ Check if student can attempt test:
 - ☐ Test is live (between start_time and end_time)

- ☐ Student is assigned to this test
 - ☐ Student hasn't already attempted (if configured)
 - ☐ Test has questions (not empty)
- ☐ Return boolean + reason if can't attempt
- **Hours 13-24:** Time tracking
 - ☐ Track time spent on each question

☐ When student clicks "Save & Next":

```
{ questionId: "uuid", timeSpent: 45 // seconds}
```

-
- ☐ Store in student_responses table
- ☐ Calculate total time taken
- ☐ Identify fast/slow questions
- **Hours 25-36:** Pause/Resume feature
 - ☐ POST /api/tests/attempts/:attemptId/pause
 - ☐ POST /api/tests/attempts/:attemptId/resume
 - ☐ Track pause duration (don't count in total time)
 - ☐ Limit pause duration (max 5 minutes total)
 - ☐ Store pause history

Deliverables Day 1-3:

- ☒ Attempt validation working
- ☒ Time tracking implemented

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Auto-Submit & Notifications

- **Hours 1-18:** Auto-submit logic
 - ☐ Build cron job to check for expired tests

☐ Run every minute:

```
// Check all IN_PROGRESS attempts where current_time > test_end_time// Auto-submit those attempts// Trigger auto-grading
```

-
- ☐ Use node-cron or Bull (job queue)
- ☐ Test thoroughly (critical feature!)
- **Hours 19-30:** Notification system
 - ☐ Set up Twilio for SMS (test account is free)
 - ☐ Set up SendGrid for email

[] Build notification service:

```
async function sendNotification(userId, type, message) { // Get user's notification
  preferences // Send SMS if enabled // Send email if enabled // Send push notification if
  enabled // Log notification in database}
```

○

- **Hours 31-36:** Test notifications

- [] Send reminder 1 hour before test starts
- [] Send notification when test is published
- [] Send notification when results are declared
- [] Test with real phone number and email

Week 3 Complete! 🎉

- Integration: Auto-submit and notifications working ✅

WEEK 4: RESULTS & ANALYTICS SUPPORT (72 hours)

Your Focus: Build supporting APIs for results and reports

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Answer Sheet & Results

- **Hours 1-15:** Answer sheet API

- [] GET /api/tests/attempts/:attemptId/answer-sheet

[] Return complete answer sheet:

```
{ "attemptId": "uuid", "studentName": "John Doe", "testName": "JEE Main Mock Test 1",
"questions": [ { "questionNumber": 1, "questionText": "...", "studentAnswer": "B",
"correctAnswer": "A", "isCorrect": false, "marksAwarded": -1, "timeSpent": 45 } ],
"totalScore": 285, "totalMarks": 300}
```

○

- **Hours 16-30:** Rank calculation

- [] After a test is submitted, calculate rank

[] Logic:

```
// Count how many students scored more than this studentconst rank = await
countStudentsWithHigherScore(testId, studentScore);return rank + 1; // rank starts from 1
```

○

- [] Update test_attempts table with rank
- [] Recalculate ranks when new submissions come

- **Hours 31-36:** Percentile calculation
 - ☐ Calculate percentile (what % of students you beat)
 - ☐ Formula: `percentile = (students below you / total students) * 100`
 - ☐ Store in test_attempts table

Deliverables Day 1-3:

- ☒ Answer sheet API ready
 - ☒ Rank and percentile calculation working
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Batch Operations & Scheduled Tasks

- **Hours 1-18:** Batch report generation
 - ☐ POST /api/reports/generate-batch
 - ☐ Accept: classId or testId
 - ☐ Generate reports for all students

☐ Use job queue (Bull) for async processing:

```
reportQueue.add('generate-batch', { classId: 'uuid', reportType: 'performance' });
```

- ☐ Process in background (don't block API)
- ☐ Notify admin when complete
- **Hours 19-30:** Scheduled tasks
 - ☐ Set up node-cron for scheduled jobs
 - ☐ Daily task (runs at 2 AM):
 - ☐ Calculate attendance percentage
 - ☐ Generate daily reports
 - ☐ Send low attendance alerts
 - ☐ Weekly task (runs Sunday 11 PM):
 - ☐ Generate weekly progress reports
 - ☐ Email to parents
 - ☐ Monthly task:
 - ☐ Archive old data
 - ☐ Generate monthly analytics
- **Hours 31-36:** Month 1 wrap-up
 - ☐ Code refactoring and cleanup
 - ☐ Final testing of all Month 1 features
 - ☐ Update all documentation
 - ☐ Month 1 demo with team

- [] Celebrate! 🎉

Week 4 Complete! Month 1 Done! 🎉🎉



MONTH 2: AI FEATURES SUPPORT (WEEKS 5-8)

WEEK 5: AI API SUPPORT (72 hours)

Your Focus: Support Aditya with AI integration testing and APIs

Day 1-3 (Monday-Wednesday) - 36 hours

Task: AI Classification Testing

- **Hours 1-18:** Test AI classification
 - [] Aditya builds classification, you test it
 - [] Prepare 100 diverse questions (different subjects, difficulties)
 - [] Run classification on all 100
 - [] Verify accuracy:
 - [] Subject correct? (Physics should detect as Physics)
 - [] Chapter correct?
 - [] Difficulty makes sense?
 - [] Document incorrect classifications
 - [] Work with Aditya to improve prompts
- **Hours 19-30:** Classification API wrapper
 - [] POST /api/questions/:id/classify
 - [] Call Aditya's AI service
 - [] Update question record with classification
 - [] Handle errors gracefully
 - [] Add retry logic (if AI API fails)
- **Hours 31-36:** Batch classification
 - [] POST /api/questions/classify-batch
 - [] Accept array of question IDs
 - [] Process in batches (10 at a time)
 - [] Show progress to user
 - [] Test with 500+ questions

Deliverables Day 1-3:

- ☒ AI classification thoroughly tested
- ☒ Batch classification working

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Similar Question Management

- **Hours 1-18:** Store generated questions

[] When AI generates similar questions, store them:

```
CREATE TABLE generated_questions ( id UUID PRIMARY KEY, source_question_id
UUID REFERENCES questions(id), generated_question JSONB, used_count INTEGER
DEFAULT 0, quality_rating INTEGER, -- 1-5 (teachers can rate) created_at TIMESTAMP);
```

- - [] POST /api/questions/generated (store generated question)
 - [] GET /api/questions/:id/similar (get similar questions)
- **Hours 19-30:** Quality validation
 - [] Build quality check for generated questions:
 - [] Must have question text
 - [] Must have correct answer
 - [] Options must be present (for MCQs)
 - [] No duplicate options
 - [] Filter out low-quality generated questions
 - [] Allow teachers to flag incorrect questions
- **Hours 31-36:** Usage tracking
 - [] Track which generated questions are used
 - [] Increment used_count when presented to student
 - [] Track effectiveness (did student get it correct?)
 - [] Week 5 wrap-up and demo

Week 5 Complete! AI Support Ready! 🎉

WEEK 6: FACE RECOGNITION SUPPORT (72 hours)

Your Focus: Support Atharav with face recognition backend

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Face Image Management

- **Hours 1-15:** Face image upload
 - [] POST /api/students/:studentId/upload-face-image
 - [] Accept base64 image or file upload

- ☐ Validate image:
 - ☐ Format: JPG or PNG
 - ☐ Size: < 5MB
 - ☐ Dimensions: min 200x200 pixels
 - ☐ Face should be visible (basic check)
- ☐ Upload to Supabase Storage
- ☐ Store URL in students.face_images array
- **Hours 16-27:** Face registration status
 - ☐ GET /api/students/:studentId/face-registration-status

☐ Return:

```
{ "isRegistered": true, "imagesCount": 8, "minRequired": 5, "lastUpdated":
"2026-01-28T10:00:00Z", "canAttemptFaceAttendance": true}
```

-
- ☐ Send reminder if < 5 images uploaded
- **Hours 28-36:** Bulk face registration
 - ☐ POST /api/classes/:classId/register-faces
 - ☐ For entire class, trigger face registration flow
 - ☐ Track progress (how many students completed)
 - ☐ Send reminders to students who haven't registered

Deliverables Day 1-3:

- ☒ Face image management working
- ☒ Registration tracking implemented

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Attendance Notification System

- **Hours 1-18:** Parent notification logic

☐ When attendance is marked:

```
async function notifyParent(studentId, status, timestamp) { // Get parent details // Send
notification: // "Your child [Name] was marked [PRESENT/ABSENT] at [Time]" // Store
notification in database}
```

-
- ☐ Batch notifications (don't send one by one)
- ☐ Send within 2 minutes of attendance marking
- **Hours 19-30:** Notification preferences
 - ☐ Let parents choose notification method:

- ☐ SMS only
 - ☐ Push only
 - ☐ Both
 - ☐ None
- ☐ POST /api/users/:userId/notification-preferences
- ☐ GET /api/users/:userId/notification-preferences
- **Hours 31-36: Attendance alerts**
 - ☐ Daily cron job: Check attendance
 - ☐ If student absent for 3 consecutive days, send alert
 - ☐ If attendance drops below 75%, send warning
 - ☐ Week 6 wrap-up

Week 6 Complete! Attendance Notifications Ready! 🎉

WEEK 7: PROCTORING SUPPORT (72 hours)

Your Focus: Support proctoring backend and violation handling

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Violation Storage & Logging

- **Hours 1-15: Violation logging**
 - ☐ POST /api/proctoring/log-violation (called from frontend)

☐ Input:

```
{ "sessionId": "uuid", "type": "TAB_SWITCH", "severity": "MEDIUM", "timestamp":
"2026-01-28T10:05:30Z", "metadata": { "previousTab": "exam", "currentTab":
"google.com" }}
```

- ☐
- ☐ Store in violations table
- ☐ Update violation count in proctoring_sessions
- ☐ Decrease trust score
- **Hours 16-27: Real-time violation alerts**
 - ☐ Set up WebSocket or Server-Sent Events (SSE)
 - ☐ When violation occurs, send real-time alert to teacher
 - ☐ Teacher dashboard shows:
 - ☐ Which students have violations
 - ☐ Type and severity
 - ☐ Live count
- **Hours 28-36: Violation threshold enforcement**

☐ When trust score < 50 OR violations >= 3:
// Auto-submit the test await submitTest(attemptId); // Notify student (test submitted due to violations) // Notify teacher

-
- ☐ Test thoroughly (critical feature)

Deliverables Day 1-3:

- ☒ Violation logging working
 - ☒ Real-time alerts implemented
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Snapshot Management

- **Hours 1-18:** Webcam snapshot storage
 - ☐ POST /api/proctoring/upload-snapshot
 - ☐ Accept base64 image (from student's webcam)
 - ☐ Upload to Supabase Storage
 - ☐ Store URL in proctoring_sessions.snapshots array
 - ☐ Compress images (reduce storage cost)
- **Hours 19-30:** Snapshot review UI backend
 - ☐ GET /api/proctoring/sessions/:sessionId/snapshots
 - ☐ Return all snapshots with timestamps
 - ☐ Allow teacher to flag suspicious snapshots
 - ☐ POST /api/proctoring/snapshots/:id/flag
- **Hours 31-36:** Cleanup old data
 - ☐ Scheduled job: Delete snapshots older than 30 days
 - ☐ Archive violation data (move to cold storage)
 - ☐ Week 7 wrap-up

Week 7 Complete! Proctoring Support Ready! 🎉

WEEK 8: OCR SUPPORT & FILE MANAGEMENT (72 hours)

Your Focus: Support paper generation with file handling

Day 1-3 (Monday-Wednesday) - 36 hours

Task: File Upload & Management

- **Hours 1-15:** Multi-file upload
 - ☐ POST /api/files/upload
 - ☐ Support multiple file formats:
 - ☐ Images (JPG, PNG)
 - ☐ PDFs
 - ☐ Word documents (.docx)
 - ☐ Validate file size (< 10MB per file)
 - ☐ Upload to Supabase Storage
 - ☐ Return file URL and metadata
- **Hours 16-27:** File conversion
 - ☐ Convert Word to PDF (use libreoffice or online API)
 - ☐ Convert PDF to images (for OCR processing)
 - ☐ Use libraries like pdf-lib or pdf2pic
- **Hours 28-36:** File organization

☐ Organize files by institute and type:

/storage /institute_uuid /question_papers /study_materials /student_faces

-
- ☐ GET /api/files/list (list files by type)

Deliverables Day 1-3:

- ☒ File upload system working
- ☒ File conversion implemented

Day 4-6 (Thursday-Saturday) - 36 hours

Task: OCR Result Processing

- **Hours 1-18:** Store OCR results

☐ Create ocr_results table:

```
CREATE TABLE ocr_results ( id UUID PRIMARY KEY, file_url TEXT, extracted_text TEXT,
confidence_score DECIMAL, status VARCHAR(50), -- PROCESSING, COMPLETED,
FAILED processed_by VARCHAR(50), -- TESSERACT, GOOGLE_VISION created_at
TIMESTAMP);
```

-
- ☐ POST /api/ocr/results (store OCR output)
- ☐ GET /api/ocr/results/:id
- **Hours 19-30:** Manual correction interface backend

- [] POST /api/ocr/results/:id/correct
- [] Allow teacher to edit extracted text
- [] Track corrections (improve OCR over time)
- [] Update extracted_text in database
- **Hours 31-36:** Month 2 wrap-up
 - [] Thorough testing of all Month 2 features
 - [] Code refactoring
 - [] Documentation update
 - [] Month 2 demo and retrospective

Week 8 Complete! Month 2 Done! 🎉🎉



MONTH 3: POLISH & LAUNCH (WEEKS 9-12)

WEEK 9: REPORTS & EXPORT (72 hours)

Your Focus: Build export features and data downloads

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Data Export Features

- **Hours 1-15:** Excel export
 - [] GET /api/export/students (export student list)
 - [] GET /api/export/test-results/:testId
 - [] GET /api/export/attendance/:classId

[] Use xlsx library:

```
const XLSX = require('xlsx');const workbook = XLSX.utils.book_new();const worksheet = XLSX.utils.json_to_sheet(data);XLSX.utils.book_append_sheet(workbook, worksheet, 'Students');XLSX.writeFile(workbook, 'students.xlsx');
```

○

- **Hours 16-27:** CSV export
 - [] Similar to Excel, but simpler format

[] Use PapaParse to generate CSV:

```
const csv = Papa.unparse(data);return csv;
```

○

- **Hours 28-36:** PDF report generation support

- ☐ Work with Aditya on PDF generation
- ☐ Prepare data for reports:
 - ☐ Student performance summary
 - ☐ Class analytics
 - ☐ Test analysis
- ☐ Test PDF generation

Deliverables Day 1-3:

- ☒ Excel and CSV export working
 - ☒ PDF report data preparation done
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Bulk Operations

- **Hours 1-18:** Bulk student import
 - ☐ POST /api/students/import-bulk
 - ☐ Similar to question import
 - ☐ Parse CSV/Excel with student data
 - ☐ Create accounts for all students
 - ☐ Send welcome emails
- **Hours 19-30:** Bulk operations management
 - ☐ Track bulk operation status
 - ☐ Allow cancellation of in-progress operations
 - ☐ Show progress (processed X of Y)
- **Hours 31-36:** Week 9 wrap-up
 - ☐ Test all export features
 - ☐ Code review
 - ☐ Demo to team

Week 9 Complete! Export Features Ready! 🎉

WEEK 10: PAYMENT SUPPORT (72 hours)

Your Focus: Support Aditya with payment integration testing

Day 1-4 (Monday-Thursday) - 48 hours

Task: Payment Testing & Webhooks

- **Hours 1-20:** Test payment flow

- ☐ Test Razorpay integration thoroughly
- ☐ Test all payment methods:
 - ☐ Credit/debit cards
 - ☐ UPI
 - ☐ Net banking
 - ☐ Wallets
- ☐ Test payment success flow
- ☐ Test payment failure flow
- ☐ Test retry logic
- **Hours 21-36:** Webhook handling support

- ☐ Set up webhook endpoint
- ☐ Verify webhook signature (security)

☐ Handle different events:

```
switch(event.type) { case 'payment.authorized': // Update subscription status break;
case 'payment.failed': // Send failure notification break; case 'subscription.activated': //
Unlock features break; case 'subscription.cancelled': // Lock features break;}
```

-
- **Hours 37-48:** Invoice generation support
 - ☐ Prepare invoice data (institute details, amounts, GST)
 - ☐ Test invoice PDF generation
 - ☐ Email invoices automatically

Deliverables Day 1-4:

- ☒ Payment integration thoroughly tested
- ☒ Webhooks working reliably

Day 5-6 (Friday-Saturday) - 24 hours

Task: Subscription Management

- **Hours 1-12:** Feature gating logic

☐ Middleware to check subscription:

```
async function checkSubscription(req, res, next) { const institute = await
getInstitute(req.user.instituteId); if (institute.subscriptionStatus !== 'ACTIVE') { return
res.status(403).json({ error: 'Subscription expired. Please renew.' }); } next();}
```

-
- ☐ Apply to premium features only
- **Hours 13-20:** Usage tracking

- ☐ Track feature usage (for analytics):
 - ☐ Tests created this month
 - ☐ Students added
 - ☐ Storage used
- ☐ Alert if nearing limits
- **Hours 21-24:** Week 10 wrap-up

- ☐ Final payment testing
- ☐ Demo to team

Week 10 Complete! Payment Support Done! 🎉

WEEK 11: TESTING & BUG FIXES (72 hours)

Your Focus: Extensive testing and bug fixing

Day 1-6 (Monday-Saturday) - 72 hours

Task: Quality Assurance

- **Hours 1-20:** Unit testing
 - ☐ Write tests for all your APIs
 - ☐ Aim for 80%+ code coverage
 - ☐ Use Jest and Supertest
 - ☐ Run tests in CI/CD pipeline
- **Hours 21-40:** Integration testing
 - ☐ Test complete user flows:
 - ☐ Student registration → face registration → take test → view results
 - ☐ Teacher creates test → assigns to students → views reports
 - ☐ Test edge cases
 - ☐ Test error scenarios
- **Hours 41-60:** Bug fixing
 - ☐ Fix all bugs found in testing
 - ☐ Prioritize critical bugs (crashes, data loss)
 - ☐ Document all fixes
- **Hours 61-72:** Performance testing
 - ☐ Test API response times
 - ☐ Optimize slow queries
 - ☐ Test with large datasets (10,000+ questions)
 - ☐ Load testing (simulate 100 concurrent users)

Week 11 Complete! App is Solid! 🎉

WEEK 12: DEPLOYMENT & LAUNCH (72 hours)

Your Focus: Support Aditya with production deployment

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Deployment Support

- **Hours 1-12:** Environment configuration
 - ☐ Set up production environment variables
 - ☐ Verify all API keys are correct
 - ☐ Test database connections
 - ☐ Verify Supabase production project
- **Hours 13-24:** Deployment testing
 - ☐ Test deployed APIs (production URLs)
 - ☐ Verify all features work in production
 - ☐ Test payment in live mode (with small amount)
 - ☐ Check performance in production

Deliverables Day 1-2:

- ☒ Production deployment verified

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Beta Testing Support

- **Hours 1-12:** Beta institute setup
 - ☐ Create accounts for beta institutes
 - ☐ Import sample data (students, questions)
 - ☐ Conduct onboarding calls
 - ☐ Provide technical support
- **Hours 13-24:** Bug monitoring
 - ☐ Monitor Sentry for production errors
 - ☐ Fix critical bugs immediately
 - ☐ Document all issues

Deliverables Day 3-4:

- ☒ Beta institutes successfully onboarded
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Documentation & Launch

- **Hours 1-12:** Final documentation
 - ☐ Complete API documentation
 - ☐ Write troubleshooting guide
 - ☐ Create FAQ document
 - ☐ Record video tutorials
- **Hours 13-20:** Launch preparation
 - ☐ Final checks (all features working)
 - ☐ Customer support setup
 - ☐ Monitor for first 24 hours
- **Hours 21-24:** Celebration! 🎉
 - ☐ We launched!
 - ☐ Team celebration
 - ☐ Reflect on learnings
 - ☐ Plan next features

Week 12 Complete! WE LAUNCHED! 🚀🎉

YOUR KEY METRICS (Track Weekly)

Development:

- ☐ APIs completed: ____ / ____
- ☐ Tests written: ____
- ☐ Bugs fixed: ____
- ☐ Code reviews done: ____

Learning:

- ☐ New skills learned this week: ____
- ☐ Blockers faced: ____
- ☐ Help received from team: ____

Quality:

- ☐ Code coverage: ____%
 - ☐ Bugs introduced: ____
-

YOUR TOOLS

Daily:

- VS Code
- Postman/Thunder Client
- Supabase Dashboard
- GitHub

Testing:

- Jest (unit tests)
- Supertest (API tests)
- Postman (manual testing)

Communication:

- WhatsApp (daily updates)
 - GitHub (code reviews)
 - Google Meet (meetings)
-

YOUR PERSONAL GROWTH GOALS

Technical Skills:

- Master Node.js backend development
- Learn Supabase deeply
- Become expert in API design
- Learn testing best practices

Professional Skills:

- Work in a fast-paced team
- Meet tight deadlines
- Code reviews and collaboration
- Problem-solving under pressure

Career Impact:

- Build a real product used by thousands
 - Strong portfolio project
 - Backend development expertise
 - Valuable startup experience
-

FINAL CHECKLIST

By Week 4:

- ☐ All user management APIs done
- ☐ Question upload system working
- ☐ Test session support ready

By Week 8:

- ☐ AI features supported
- ☐ Face recognition backend complete
- ☐ Proctoring system working

By Week 12:

- ☐ All features tested
 - ☐ Production deployment successful
 - ☐ Beta institutes onboarded
 - ☐ YOU LAUNCHED A PRODUCT! 🚀
-

RUSHIKESH'S MANTRA

"I am not just supporting, I am building."

Every API you write enables:

- Teachers to manage better
- Students to learn better
- Institutes to run efficiently

Daily Affirmations:

- "I ask questions when I don't understand (smart, not weak)"
- "I test my code thoroughly before pushing"
- "I help my teammates when they need me"
- "I learn something new every day"
- "I rest on Sundays to recharge"

Remember:

- You're learning while building (incredible opportunity!)
- Aditya is there to help you (ask questions!)
- Your code will be used by thousands
- Every bug you fix makes the app better
- You got this! 💪

LET'S BUILD SOMETHING AMAZING, RUSHIKESH! 🚀

Next Steps:

1. Read this document fully
2. Clarify doubts with Aditya
3. Set up environment (Day 1)
4. Start coding! (Day 2)

Believe in yourself! You're going to learn and grow so much! 🙌